

BusGuard passenger assistance system

Singapore BusTech Grand Challenge 2026

IHL Student Category

Project report

29 September 2026

DRAFT FOR TEAM AND STAFF REVIEW

Institution Singapore Polytechnic [TO CONFIRM]

Team name [TO CONFIRM]

Team members [ALL STUDENT NAMES TO CONFIRM]

Staff in charge [TO CONFIRM]

This report describes the current BusGuard prototype, its software verification and the remaining physical and passenger validation. The supplied institution logo suggests Singapore Polytechnic; the team must confirm the cover details.

Working CCTV and RFID inputs support the prototype. Doors, ramp operation and vehicle movement are simulated. The budget is a proposed allocation pending confirmation.

Before submission, complete the title details, confirm the budget and contribution record, and obtain staff review. The official brief allows one report e-submission per team through the staff in charge.

Abstract

Autonomous public transport must support passengers who need additional boarding time, ramp access, priority seating or clearer instructions. BusGuard investigates a connected assistance workflow that combines two camera views, RFID boarding records and explicit passenger requests. One local controller coordinates a detection console, a passenger website and an interactive 3D bus. The inside camera counts people throughout the journey and checks for standing after door closure; outside activity influences the boarding window. Registered RFID cards identify declared passenger profiles and record boarding or alighting. A passenger can request one assistance option, follow its countdown and receive visual, spoken or supported-device vibration feedback. The controller separates a 50-second admission cutoff from completion of already accepted allowances, and requires five continuous seconds of fresh negative standing evidence before simulated departure. The September 2026 source-publication checks recorded 936 passing Python tests, 307 passing JavaScript tests and a successful viewer build. These results verify software behavior, not camera accuracy or passenger satisfaction. Cameras and RFID are physical inputs; movement, doors, ramp and securement confirmation remain simulated. The prototype demonstrates coordinated assistance decisions while identifying the perception, usability and mechanical validation required before real deployment.

1 Introduction background and purpose

The competition brief asks for an integrated Autonomous Passenger Assistance System combining sensors, machine learning and automated mechanical systems, with reliable recognition, appropriate responses and measurable accessibility and satisfaction improvements.[1] Without a driver or service operator, a passenger may have difficulty communicating a need, understanding whether assistance was accepted, or knowing when it is appropriate to board. A camera detection alone does not resolve these interaction and coordination problems.

BusGuard addresses the connection between observation, request, decision and feedback. It provides a shared operational state rather than three independent demonstrations. A passenger request on a phone can change the controller's assistance queue, its allowance and the access sequence shown by the bus model. An RFID tap updates a registered card's onboard status. Camera evidence supports observation and departure checking without being treated as an identity or diagnosis.

The project makes three design distinctions. First, visible objects, declared profiles and explicit requests carry different information. A wheelchair or walking aid can indicate a potential assistance requirement, while pregnancy, age and sensory needs must not be inferred from appearance. Second, a boarding deadline is not permission to depart: a valid standing check and other holds must still be resolved. Third, an accepted app request is not proof that a person has physically boarded.

The current objective is a repeatable local-network prototype using CCTV and an ESP32/RC522 reader, with web-based interaction and simulated vehicle responses. Its contribution is the connected workflow,

timing and evidence handling. It does not claim a new foundational vision model, verified autonomous driving, or completed physical ramp integration. The report therefore separates implemented behavior, available software evidence and evaluation still required.[2,3]

2 Methods and system design

2 1 Shared controller and three interfaces

A Python/FastAPI service owns the bus state, passenger requests, RFID registry and timing rules. The detection console runs on port 4479, the 3D viewer on 4480, and the passenger site on 4481. An optional HTTPS passenger endpoint on 4482 supports phone microphone access when a matching certificate is trusted. All devices must reach the same local server. The viewer and passenger sites expose restricted interfaces to the primary controller rather than maintaining separate journey decisions.[2]

The controller updates approximately every 200 milliseconds, independently of whether a dashboard is open. Its main data paths are:

Input	Processing	Shared result
Outside live camera	Object detection, freshness validation and assistance-event deduplication	Boarding activity and possible assistance request
Inside live camera	Ordinary person count, temporal vote, cabin audit and separate standing pass	Occupancy estimate and departure evidence
RFID reader	Reader authentication, UID lookup, admission validation and retry protection	Onboard-card state and count adjustment
Passenger app	Location, capacity, timing and request-ownership checks	One accepted assistance choice and status/countdown
Operator controls	Explicit scenario and hold actions	Arrival, reset, obstruction or simulated mobility confirmation

The Singapore-inspired 3D bus has six priority and twenty standard seats, two doors, camera representations, an entrance information display and a movable status panel. Animated wheels and road movement communicate travelling; door and ramp animation follows controller state. Opening the viewer on another device transfers its rendering work to that device without duplicating inference. The A-E route is schematic and operator-triggered, not GPS navigation.



Figure 1 Current 3D bus interface captured from an isolated software preview. The image illustrates the model and does not represent a physical actuator or live-camera validation.

2.2 Perception and processing latency

The default hybrid backend uses YOLOE for ordinary object categories and a separate standing-person prompt, with Grounding DINO for detailed descriptive phrases. Per-category confidence settings and a separate standing threshold are available. These are pretrained components; no bus-specific fine-tuning or measured cabin accuracy is claimed. Phrase detection remains an experimental inspection feature and is not a replacement for the literal person category required for cabin counting.[2,4]

Tracking and adaptive box smoothing reduce visible jitter. A bounded dispatcher serializes model access and replaces pending work from the same source instead of accumulating an unlimited queue. RTSP capture continuously decodes frames while inference consumes recent available frames. Frame identities and source generations reject obsolete results. The YOLOE adapter also reuses a supported predictor when switching ordinary and standing prompts, avoiding repeated model preparation.

These choices address avoidable processing backlog but cannot remove camera encoding delay, Wi-Fi buffering or device limits. Inference duration is not the same as scene-to-screen delay. Reliable evaluation must timestamp the real scene event, capture, result publication and display separately. No fixed real-time frame rate or deployment-wide latency result is asserted in this report.

2 3 Different roles for inside and outside cameras

Outside inference operates only when the bus is stationary and the doors are fully open. Fresh positive outside observations can renew the quiet boarding window. Repeated polls, predicted boxes and uploaded test media are not new boarding activity. Assistance-related object events are deduplicated before they can become requests; a repeated wheelchair box should not continuously create unrelated passenger requests.

Inside person counting continues during boarding and travel while the source remains connected. Standing inference is paused during opening, open, closing or unknown door states and starts only after fully closed doors, when enabled. A door-generation token prevents an earlier frame that finished late from becoming post-closure evidence. Uploaded images and recordings can inspect detections but cannot authorize live departure.

The current standing implementation uses the text prompt standing person; it does not classify sitting or reconstruct MediaPipe joints. An unmatched positive standing box still holds departure. Conversely, valid negative standing evidence does not require an old set of passenger track IDs to reappear. This separates posture checking from tracking churn and historical occupancy.

2 4 Count stabilization and reconciliation

The visible object count uses a rolling, time-weighted one-second vote. If seven people are observed for 700 milliseconds, six for 200 milliseconds and five for 100 milliseconds, the modal count is seven with 70% temporal support. The default acceptance threshold is 65%, with a complete initial window. Support expresses consistency over observed time, not the calibrated probability that the count is correct.

Occupancy reconciliation operates on a longer scale. With whole-cabin coverage confirmed, each consecutive one-second bucket retains its latest fresh stable person count. After thirty complete buckets, the arithmetic mean is rounded to the nearest integer, with a half rounded upward. A missing or stale sample abandons the partial cycle rather than supplying zero. Thus a high camera frame rate does not overweight one second and a disconnected camera does not create an empty bus.[2]

Accepted RFID boarding and alighting events adjust the record by +1 and -1. A completed camera audit corrects the population baseline; it is not added as a second population. Later taps adjust that corrected baseline. Count logic also preserves valid taps around the audit completion boundary. Active assistance requests can reserve availability, but submitting or completing a request does not add a second passenger.

Available seats are derived from a capacity of 26, recorded allocations, reservations and corrected occupancy. Priority/standard distribution is estimated; no camera identifies which physical seat each passenger occupies. Ordinary passengers may use spare priority seats. A priority profile can trigger

reassignment within the simulation and a polite request for passengers who do not need priority seating to move to an available standard seat.

2.5 Stop timing and departure logic

The operator starts the scenario and subsequently presses Stop to trigger arrival at the next station on the A-B-C-D-E loop. An approach announcement precedes a simulated slowdown. The stop clock begins when the doors are fully open. The initial no-activity window is fifteen seconds, so a quiet stop does not automatically wait for fifty seconds.

Let t denote seconds after the doors fully open and D the door-open deadline. Initially $D = 15$. The current rules are:

Accepted event	Deadline update
Fresh outside detection	Later of existing deadline and $\min(50, \text{detection time} + 15)$
RFID boarding or alighting tap	Later of existing deadline and $\text{tap time} + 10$
Ordinary passenger assistance request	Later of existing deadline and $\text{request time} + 10$
Ramp request	Later of existing deadline and $\text{request time} + 20$

Inside detections do not extend boarding. Allowances are measured from acceptance, not blindly stacked onto an already longer deadline. New requests and taps are rejected from fifty seconds onward, while an allowance accepted earlier can finish: a tap at 49 seconds can protect access until 59 seconds, and a ramp request at 49 seconds until 69 seconds. A rejection does not change occupancy or reopen admission.

When the deadline expires, unfinished requests expire, the ramp stows if necessary, and doors close if the doorway is clear. A ten-second departure notice is scheduled against the current plan; it is never backdated. Extensions or holds can postpone movement. A countdown is therefore a plan, not an unconditional departure promise.

After closure, departure requires five continuous seconds of fresh, successful standing checks with no standing detected. Standing, source replacement, stale frames, disconnection or invalid results reset confirmation. Repeated polls cannot advance it. The three-second minimum closed-door interval overlaps this check; it is not an additional wait afterwards. Departure also requires other holds to clear, including emergency, doorway obstruction, overcapacity, active assistance and mobility review.

No-standing evidence is not proof that everyone is seated. The detector can miss a person. While already moving, standing evidence produces a queued seating reminder rather than sudden simulated braking. Server restart also requires explicit revalidation/reset; old countdowns and camera confirmations are not resumed.

2 6 RFID integration

The ESP32 connects to an RC522 over SPI: SCK 18, MISO 19, MOSI 23, chip select 21 and reset 22, with 3.3 V and ground. No external LED or buzzer is required. The firmware transmits a card UID, reader identity, unique event ID, current window ID and recent observation timestamp to the server. Reader credentials are separate from operator pairing and the OpenAI key.[2]

Four card slots have configurable normal, senior, pregnant or custom profiles. UID mappings are preconfigured; the console edits profiles, not new-card enrollment. A first accepted tap records boarding and a subsequent accepted tap of that onboard card records alighting. Scans require open doors and an eligible admission window. Unknown, stale, duplicate and rapidly repeated presentations do not silently toggle occupancy. Exact retries return the earlier outcome.

The reader communicates passenger-declared/profile information rather than detecting demographic characteristics. A senior or pregnant profile triggers a polite priority-seat announcement. UIDs are identifiers, not secure personal identity or fare-payment credentials. Private reader tokens and real card IDs are excluded from the shared source export.

2 7 Passenger interaction and optional voice

The passenger page presents assistance commands first, with a compact route, seat availability and settings. Each new request selects one option: ramp, extra time, audio guidance, visual guidance or priority seating. Boarding requests require the passenger's selected demo location to match the bus's current stop. Alighting uses the On the bus location and the reached stop. In the timed scenario, admission must also be open. These are demonstration settings, not verified geolocation.

The app displays server-owned request progress and remaining allowance. Completion is explicit. Ramp use creates a separate simulated mobility-review requirement because camera posture cannot confirm restraint or securement. Arrival reminders are independent: a reminder can target a later stop without reserving a seat or extending dwell. Visual, spoken and optional vibration feedback depends on browser support and preferences; background delivery and iPhone vibration are not guaranteed.

The optional OpenAI Realtime assistant activates after “Hey BusGuard” or a deliberate talk control. It can retrieve current status, request or cancel assistance, confirm the passenger's stated completion, set reminders and change supported app preferences. It cannot move the bus or bypass holds. The server retains the API key. While listening, recognition uses cloud audio processing; the wake phrase is not an offline local engine.

The current voice mode finishes a reply before accepting another turn. Microphone input pauses while the reply is prepared and played, then resumes for a three-second follow-up window. Replies default to one short sentence, with another only when necessary. Voice failure leaves text controls available. Phone microphone use requires trusted HTTPS, browser consent and Internet/API availability.

3 Results and evidence

The available results are implementation and automated software checks. The source-publication record dated 29 September 2026 reports the following:[3]

Check	Recorded result	What it establishes
Python suite	936 passed; two warnings	Controller, API, timing, counting, RFID and related software cases exercised
JavaScript suite	307 passed	Browser-state, passenger, viewer and voice-turn cases exercised
Three.js viewer rebuild	Passed	The published browser source built with pinned dependencies
Source manifest verification for this report	245 listed files matched their SHA-256 hashes	The reviewed export matches its manifest at verification time

The Python checks used the existing development environment. JavaScript dependencies were installed separately in the source export. The test counts describe the final passing run. They do not establish that a completely new computer installation, live camera trial, physical RFID run or live voice conversation has passed. The validation record retains the initial-run failures and fixes for reproducibility.

Representative tests cover five-second standing confirmation, interrupted evidence, quiet-stop timing, late allowances, duplicate RFID receipts, capacity and request ownership/location. They show that specified inputs produce expected software outcomes. They do not show how frequently a camera recognises a real wheelchair or standing person, whether a physical RFID demonstration has passed on every target device, or how satisfied passengers are.

The physical hardware available for the intended demonstration comprises cameras and an ESP32/RFID reader; end-to-end hardware rehearsals must be recorded separately. Doors, ramp, motion and securement remain simulated. No current labelled cabin accuracy, controlled accessibility improvement or

passenger-satisfaction dataset supports a numerical claim. These are explicit outstanding results against the competition's success criteria, particularly its mechanical-response and measured-user-benefit expectations.[1]

3 1 Four demonstration cases

Case	Procedure and expected observable result	Evidence to capture
Ramp-assisted boarding	Eligible passenger requests ramp; controller grants 20 seconds; viewer opens access; completion and explicit mobility review precede departure checks	Synchronized request, console and viewer recording; accepted event times
Slower boarding with priority RFID	Registered priority card taps while admission is open; occupancy increments once; priority announcement plays; late tap retains its accepted 10-second allowance	Reader receipt, count before/after, deadline and post-cutoff rejection
Assisted alighting	Onboard passenger requests assistance at the reached stop; countdown and progress appear; their registered card's accepted next tap records alighting	App status, card direction, occupancy decrement and completed access cycle
Standing and failure handling	Stand after closure, then sit; show reset followed by five fresh clear seconds; disconnect camera or assert obstruction to show a distinct hold	Frame/result ages, gate progress, hold reason and recovery sequence

These are reproducible demonstration procedures, not a statement that all four physical runs have already passed. Rehearsal records should identify source mode, software snapshot, settings, operator, date and any failed attempts.

4 Proposed prototype budget

This proposed cash budget assumes the team reuses its existing cameras, reader, cards, computer and demonstration devices. It is a planning allowance of SGD 144, not a record of expenditure or supplier quotations. Asset ownership, actual purchase prices, approved seed funding and reimbursement

conditions remain to be confirmed. Existing equipment still has an economic value and is listed separately from new cash needs.

Item	Quantity or basis	Proposed cash SGD	Basis to confirm
Inside and outside CCTV	2 existing units assumed	0	Record model, ownership and original value
ESP32, RC522 and RFID cards	1 board, 1 reader, 4 cards	0	Reuse assumed; check receipts and availability
Server laptop, phone and viewer device	Existing or loaned equipment	0	Record specifications and in-kind value
Spare cables and network accessories	Rehearsal allowance	20	Itemise required replacements
Optional voice API usage	Capped rehearsal and event allocation	30	Configure usage controls and record billed usage
Poster printing	Planning allowance for one poster	50	Confirm print dimensions, supplier quote and payer
Demonstration materials	Labels, mounting and consumables	20	Confirm actual item list
Contingency	20 percent of SGD 120 cash subtotal	24	Staff approval required
Total proposed additional cash	Excludes existing equipment value	144	Provisional allocation, not approved spending

These amounts are proposed spending limits rather than claims about current retail prices. If any assumed existing device must be purchased, its quoted cost must be added before budget approval. The seed-money amount and final reimbursable total are unknown. Actual expenditure should record quantity, supplier, date, unit price, tax and receipt. Physical ramp and door actuators are not included in this

current prototype budget; adding them requires a separately costed mechanical design, feedback sensors and validation plan.

5 Discussion and evaluation plan

5 1 Design choices and alternatives

A camera-only approach can observe visible objects but cannot reliably discover every assistance need. An app-only approach excludes passengers unable or unwilling to use a phone. RFID alone records only registered presentations and cannot detect an unrecorded cabin occupant. Combining channels is useful because each has a stated role and limitations; agreement is not manufactured by summing them.

Time voting reduces flicker, while the longer camera mean checks the ledger. Neither method can correct persistent systematic misclassification. The delayed mean also smooths genuine changes, so immediate accepted RFID events remain important. Standing-only prompting avoids requiring seated skeletons for every tracked person, but sacrifices positive seated classification. This tradeoff must be judged using representative held-out footage rather than preference for a faster model alone.

Local processing limits dependence on external services for core decisions. Optional cloud voice improves interaction possibilities but introduces cost, connectivity and privacy considerations. The visual/text workflow must remain usable when voice is unavailable. Explicit request intent and profile declarations also avoid presenting demographic guesses as accessibility knowledge.

5 2 Controlled evaluation still to be completed

Before collecting data, freeze the software snapshot, camera placement, resolution, prompts and thresholds. Separate threshold-development examples from held-out evaluation. Record false positives and false negatives, not just successful detections, and obtain consent for recordings or participation.

Evaluation	Proposed controlled method	Measures; not current results
Standing recognition	At least 60 short held-out clips spanning standing, seated and partially occluded/leaning cases; label by two reviewers and resolve disagreements	Precision/recall, false alarms, missed-standing rate and uncertainty by scene condition
Passenger counting	Staged known occupancy changes with both cameras active; include empty, partially occluded and moving participants	Absolute count error, error after 30-second correction, incomplete-cycle frequency

Evaluation	Proposed controlled method	Measures; not current results
Timing and RFID	Script quiet stops, taps near cutoff, duplicate retries, four registered cards, unknown cards and disconnect/reconnect events	Accepted/rejected event correctness, deadline error, double-count incidents and recovery time
End-to-end response	Record real scene events and synchronized timestamps under one- and two-camera load	Median and 95th-percentile scene-to-result/feedback delay; frame age and queue time
Accessibility/usability	Small consented pilot with representative users where practicable; compare the same tasks with and without guided requests, counterbalancing order	Task completion, time, errors, assistance needed and a defined post-task rating

Sample sizes above are proposed planning targets, not performed trials or a claim of statistical sufficiency. Report participant characteristics only with consent and avoid treating simulated disability as equivalent to lived experience. Decide success thresholds before testing, report all exclusions, and state whether results generalize beyond the tested view/devices. A satisfaction claim requires an actual instrument, response count and results.

5 3 Contribution and provenance

The project builds on publicly available models and software: YOLOE, Grounding DINO, BoT-SORT, the One Euro filtering method, Three.js, FastAPI, OpenAI services and embedded libraries. Their algorithms, pretrained weights and rendering/runtime foundations are not claimed as original work.[4] Passenger-request and assistance-progress ideas were informed by the BusTech repository. The project acknowledges that design influence.[5]

The project-specific work is the integrated inside/outside policy, separate standing and count decisions, admission/allowance logic, RFID retry/accounting behavior, passenger-interface coordination and state-driven bus presentation. Development used AI-assisted coding and documentation. The final contribution record must name each student's actual work and distinguish original integration, adapted ideas and third-party components. The competition explicitly evaluates only substantial new original elements when improving another design.[1] The team should confirm attribution and applicable institutional disclosure rules before signing off.

6 Conclusion

BusGuard implements a connected passenger-assistance prototype in which cameras, RFID and explicit requests feed one controller and three coordinated websites. Its principal engineering result is a clear separation between observing activity, accepting assistance, estimating occupancy and permitting simulated departure. The current workflow supports short quiet stops, bounded admissions with completed accepted allowances, five-second fresh standing checks and intelligible failure holds.

Automated verification supports the software rules, with 936 Python tests, 307 JavaScript tests and a successful viewer build recorded for the source export. It does not establish perception accuracy, passenger satisfaction or real-vehicle readiness. The immediate priorities are complete hardware rehearsals, labelled camera evaluation, representative usability feedback, confirmed budget/attribution and an explicit explanation of the simulated mechanical scope. Physical ramp/door automation and validated securement feedback remain future engineering work. Presenting these boundaries transparently makes the prototype's contribution and next steps assessable against the competition brief.

Bibliographic references

- [1] Singapore Bus Academy / Land Transport Authority. Singapore BusTech Grand Challenge 2026 - IHL Student Category: Updates 18 August 2026. User-supplied briefing PDF. Task and success criteria: pages 8-13; judging: page 16; submission timeline and report format: pages 17-18. Dates/venues are marked tentative in the briefing.
- [2] BusGuard project source export, BusGuard-2026-09-29. Reviewed implementation and documentation: vision/assistance.py, scenario.py, departure.py, standing.py, cabin_audit.py, rfid.py; docs/SYSTEM_WORKFLOW.md, ARCHITECTURE.md, API_REFERENCE.md; passenger and bus source; firmware/BusGuardRFID. Source manifest contains 245 files; SHA-256: e35f65ac1913a7b67c3d81cf4bdc648f67c73b8375183bfb108d1f3f391f0. Repository: <https://github.com/hpone2k/BusGuard> (private). Verified commit 6140e20511f7279307e670b2869f82dda10dd03b. Access requires permission from the repository owner.
- [3] BusGuard. Source publication checks - 29 September 2026, docs/VALIDATION_CURRENT.md in the reviewed export. Records test/build results and their scope and qualifications; not a physical-device or accuracy study.
- [4] BusGuard. Third-party components, THIRD_PARTY_NOTICES.md in the reviewed export, with upstream references: Ultralytics, YOLOE documentation (<https://docs.ultralytics.com/models/yoloe/>); IDEA Research, Grounding DINO (<https://github.com/IDEA-Research/GroundingDINO>); Nir Aharon et al., BoT-SORT (<https://github.com/NirAharon/BoT-SORT>); Géry Casiez, Nicolas Roussel and Daniel Vogel, One Euro filter (<https://gery.casiez.net/1euro/>); Three.js (<https://github.com/mrdoob/three>). These are dependency/design attributions, not evidence of BusGuard's measured performance.
- [5] BusTech repository contributors. BusTech repository (<https://github.com/zerefdragneel4356-afk/BusTech>), including the locally supplied project handoff and passenger-request concepts. Reference snapshot: commit 3062a18, dated 22 September 2026. The team must confirm attribution and the extent of any code reuse before submission.
- [6] Ultralytics. YOLOE open-vocabulary detection and segmentation documentation. <https://docs.ultralytics.com/models/yoloe/> (accessed 29 September 2026). Used for model interface context, not BusGuard accuracy claims.
- [7] IDEA Research. Grounding DINO Tiny model card. <https://huggingface.co/IDEA-Research/grounding-dino-tiny> (accessed 29 September 2026).
- [8] Miguel Balboa and contributors. Arduino RFID library for MFRC522. <https://github.com/miguelbalboa/rfid> (accessed 29 September 2026).
- [9] OpenAI. Getting started with the Realtime API. <https://developers.openai.com/api/docs/guides/realtime/> (accessed 29 September 2026).